

Reinforcement Learning (RL)

- method to train agents that can act autonomously, the crucial difference to supervised learn.
 - sup. learning only makes predictions, RL uses predictions to act, i.e. influence environment
 - supervision is usually sparse $\Rightarrow$ feedback only given after multiple actions
extreme case: 1 bit of information ("win / loss") after an entire game
(supervised: teacher explains after each action why it was good or bad)
 - setting:
 - agent acts in an "environment" that evolves in discrete time steps: $s_t \rightarrow s_{t+1}$
 - feedback is given in form of rewards r_t : $r_t > 0$: positive feedback, gain, $r_t < 0$: loss, penalty
 $r_t = 0$ (most often): no feedback for the last action
- [remark: "reward shaping" adds additional rewards during training, but must be done carefully to not bias the action relative to official rewards
example (chess): "capture the opponents queen" usually makes sense, but can be a trap - "deliberate queen sacrifice"]
- theory: extra rewards may only depend on present state s_t , not on the history (how we got there)

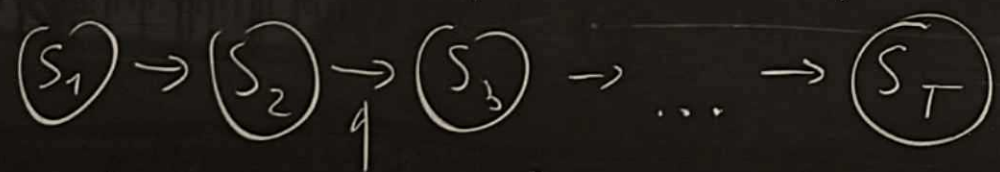
- the state of the environment may be fully visible (e.g. chess, Go) $\Rightarrow O_t = S_t$
 or can be partially hidden (e.g. card games) $\Rightarrow \overset{\uparrow}{\text{observations}} O_t \neq S_t$
 (we only cover fully visible games)

- the agent receives O_t and decides about action a_t according to some policy $\pi(O_t) \rightarrow a_t$
 decision can be deterministic (always same action in state O_t) or stochastic

Markov assumption: state S_{t+1} only depends on the present (S_t, a_t) , not on the past $(S_{t-1}, S_{t-2}, \dots, a_{t-1}, a_{t-2}, \dots)$
 (example: chess with 3 exceptions: castling, capture "en passant", draw by repetition)

draw transitions as a graph:

- Markov chain: only evolution of the environment, no actions



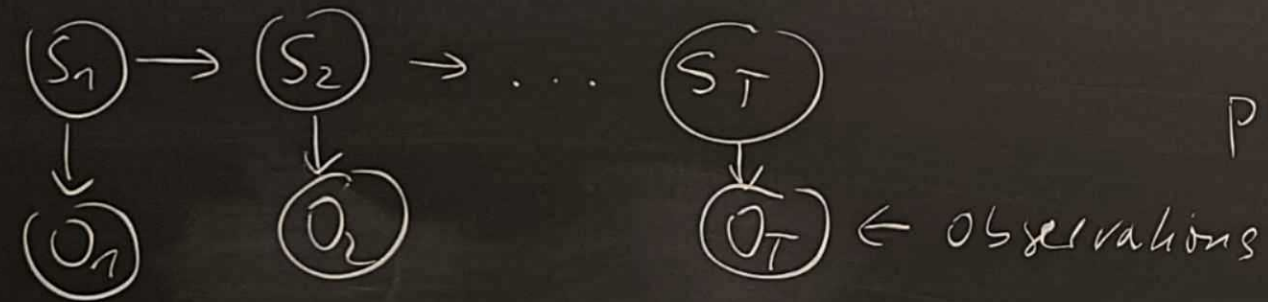
transition probabilities $p(S_3 | S_2)$

$\rightarrow$ Markovian $\hat{=}$ joint probability factorizes: $p(S_1, \dots, S_T) = p(S_1) \prod_{t=1}^{T-1} p(S_{t+1} | S_t)$
 (similar to i.i.d. assumption $p(TS) = \prod_{i=1}^N p(X_i)$)

compare to triangular maps:

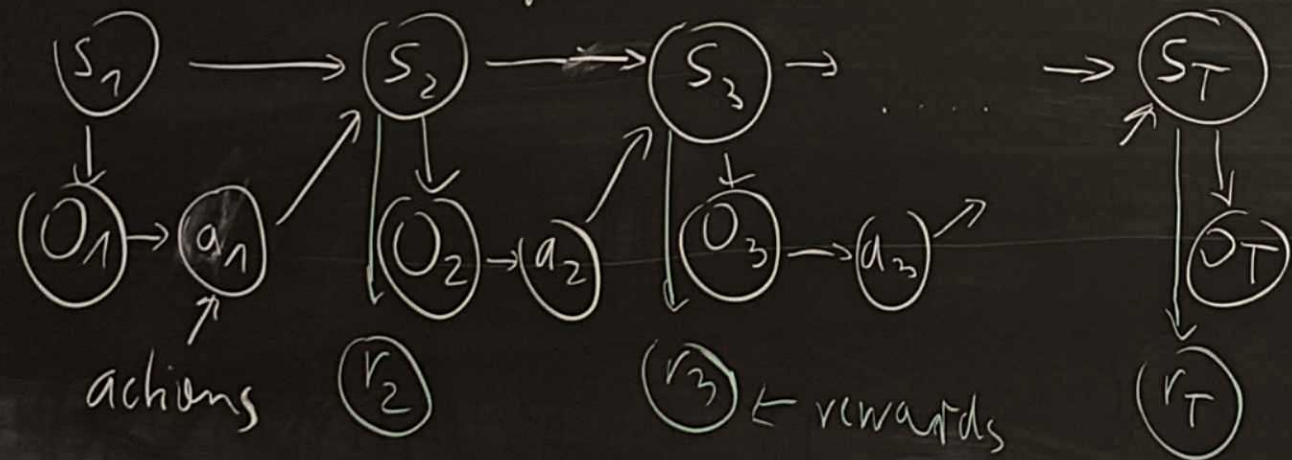
$$p(S_1, \dots, S_T) = p(S_1) \cdot \prod_{t=1}^{T-1} p(S_{t+1} | S_{\leq t})$$

- Hidden Markov Model (HMM): state is only partially observable: $o_t \sim p(o_t | s_t)$



$$p(s_1, \dots, s_T, o_1, \dots, o_T) = p(s_1) \cdot p(o_1 | s_1) \prod_{t=1}^{T-1} p(s_{t+1} | s_t) p(o_t | s_t)$$

- Markov Decision process (MDP):



transition probabilities: $p(o_t | s_t)$ observations
 $\pi(a_t | o_t)$ policy for actions
 $p(s_{t+1}, r_{t+1} | s_t, a_t)$ "world probability"

joint prob $\hat{=}$ product of all those

- simplification: fully observed MDPs: $o_t = s_t \Rightarrow$ policy $\pi(a_t | s_t)$, world probs $p(s_{t+1}, r_{t+1} | s_t, a_t)$
- two approaches:
 - + model-based RF: true world probs p^* is known or $\hat{p}$ is learned explicitly
 - model-free RF: only have training data from p^* , but do not know it and do not learn $\hat{p}$ (sometimes "transduction")

• return after time t : total reward received between t and the end T

discounted return: downweight rewards in the far future:

two effects:

- focus more on surviving the next moves than winning
- makes games learnable that have no end $T \rightarrow \infty$

$$g_t = \sum_{t'=t+1}^T \gamma^{t'-(t+1)} r_{t'}$$

$$g_t = \sum_{t'=t+1}^T r_{t'}$$

γ discount factor $\gamma \in (0, 1]$

• goal of RF: learn the optimal policy: $\pi^* = \arg \max_{\pi} \mathbb{E}_{p^*} [g_1]$

abbreviation: value function

$$V_{\pi}(s_t) = \mathbb{E}_{\pi} [g_t | \text{start in } s_t]$$

$$\mathbb{E}_{\pi} [g_1]$$

expected reward when using policy π

crucial property of an optimal policy: self-consistent: if π^* was optimal in state s_t , it should still be optimal in the next state s_{t+1}

$$\begin{aligned}
 V_{\pi}(s) &= \sum_a \pi(A=a|S=s) \cdot \sum_{s',r} p^*(S'=s', R=r|S=s, A=a) (r + \gamma g_{s'}) \\
 &= \mathbb{E}_{\pi} \left[\mathbb{E}_{p^*} \left[g_s = (r + \gamma \underbrace{\mathbb{E}[g_{s'}]}_{=V_{\pi}(s')}) \right] \right] \quad \begin{aligned} g_s &= \sum_{t'=t+1}^T \gamma^{t'-(t+1)} r_{t'} \\ g_{s'} &= \sum_{t'=t+2}^T \gamma^{t'-(t+2)} r_{t'} \end{aligned}
 \end{aligned}$$

remaining return after move a

⇒ Bellman equations

$$V_{\pi^*}(s) = \sum_a \pi^*(a|s) \sum_{s',r} p^*(s',r|s,a) \cdot [r + \gamma V_{\pi^*}(s')]$$

$V_{\pi^*}(s)$: vector of expected return values

if p^* and π^* are known, this is a linear system of equations for $V_{\pi}(s)$ when number of states and actions is finite

- p^* is known $\Leftrightarrow$ only possible in model-based RL
- if there are many states $\Rightarrow$ linear system is huge $\Rightarrow$ prefer iterative linear solvers typically "value iteration":
 - ① define initial guess $V_{\pi}^{(0)}(s)$ for all s

② for $\tau = 1, \dots, \tau_{\max}$

③ for all s :

$$V_{\pi}^{(\tau)}(s) = \max_a \sum_{s', r} p(s', r | s, a) (r + \gamma V_{\pi}^{(\tau-1)}(s'))$$

approximate this expectation
by the single term (s', r, s, a) from TS

guaranteed to converge for any policy π
 similar to the well-known Gauss-Seidel algorithm for linear systems
 even better: converges to optimal value function V_{π^*} even if trained with a suboptimal $\pi \neq \pi^*$
 $\Rightarrow$ define optimal (deterministic) policy after convergence: $\pi^*(s) = \arg \max_a \sum_{s', r} p^*(s', r | s, a) (r + \gamma V_{\pi^*}(s'))$

• for most games, we don't know p^* and don't want to learn $\hat{p} \approx p^*$ (too complicated & not necessary)

⇒ Q-learning (main example for model-free RL)

replace the Value function $V_{\pi}(s)$ with the action-value function $Q_{\pi}(s, a)$

$Q_{\pi}(s, a)$: expected return when we play a in state s , and then follow policy π to the end

[$V_{\pi}(s)$ — || — when we use policy π throughout $\Rightarrow V_{\pi}(s) = \max_a Q_{\pi}(s, a)$]

this simple trick allows to eliminate p^* from the Bellman equations / algorithm